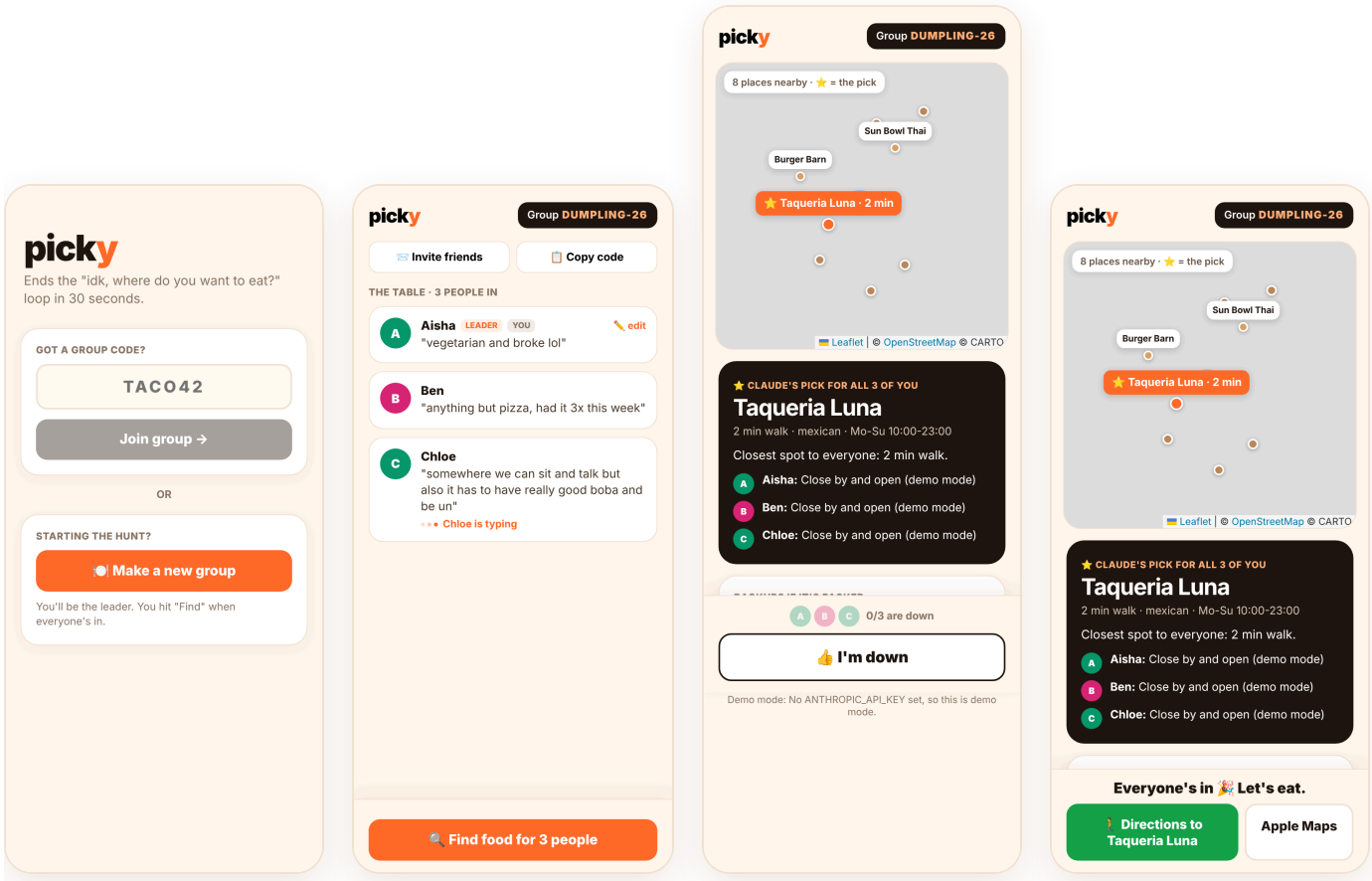
HACK WITH CLAUDE · UC BERKELEY · NOV 13, 2026

picky

Ends the "idk, where do you want to eat?" loop in 30 seconds. Everyone pitches, and Claude picks the one place that works for the whole group.



TRACK
01 · Agents in Action

BUILT BY
Smaran Kudapa

STACK
Next.js · Claude API · Upstash Redis · OpenStreetMap · Leaflet · Vercel

1. The problem

Every friend group has the same 15-minute standoff before eating: "idk, what do you want?" Everyone has constraints (one is vegetarian, one is broke, one refuses pizza, one has to be back by 9), but nobody wants to be the one to decide. The group chat goes in circles, and you end up somewhere nobody really wanted.

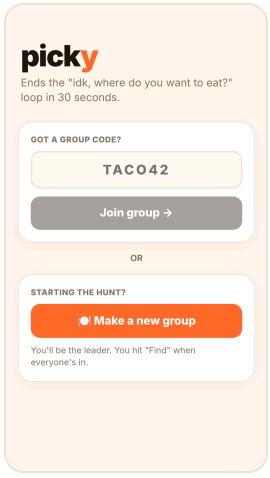
Who it's for: students and friend groups. It's a huge, non-technical audience that eats together every day. There's nothing to download. You open a link on your phone.

One-line pitch: Friends each type what they want on their phone. picky checks the real restaurants open near you and picks the one place that works for everyone, then gets you there.

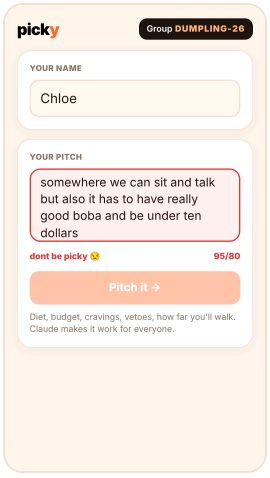
2. Why it's more than a Claude prompt

What picky adds	Why a single chat can't do it
Everyone's input at once	Each friend pitches from their own phone, live. No single person has to collect and paste everyone's constraints.
Live, real places	The app pulls the actual restaurants near the leader's GPS location (OpenStreetMap), with hours and distance.
Shared, synced state	Typing indicators, pitches locking when the leader hits Find, and a group thumbs-up vote, all synced every second.
A map and an action	Every nearby place is plotted, the pick is starred, losers are crossed out with the reason, and it ends with one-tap walking directions.

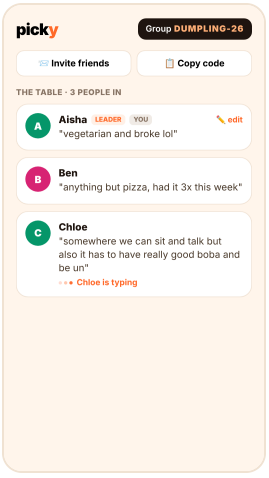
3. How it works



1 · Start or join
Enter a group code, or make a new group and become the leader.



2 · Pitch
80-char limit. Go over and the box turns red: "dont be picky 😒".



3 · Wait for friends
Pitches join one by one, with live "is typing". Anyone can edit until Find.



4 · Find
All nearby places on a map. Claude's pick is starred, with a reason for each person.



5 · Everyone's in
When all thumbs are up, it offers walking directions.

Screenshots are from a local test with 3 simulated phones. In that test the app ran in demo mode with made-up restaurants and no internet (so the map tiles are blank). The deployed app uses Claude and real map data.

4. Feature list (MVP)

Landing

- Two parts: **join with group code** or **make a new group**
- Fun codes like `TACO-42`. Case and dashes don't matter.
- Invite links (`/?g=TACO42`) pre-fill the code
- Your session survives a page refresh

Pitch

- Name and pitch
- **80-character limit**: the box turns red, shakes, and says *"dont be picky 😒"*. The button turns off and the server never saves an over-limit pitch.

Waiting room

- Friends' cards appear one by one as they join
- **"Chloe is typing"** with animated dots, live on every phone
- Edit your pitch any time until the leader hits Find
- Invite and copy-code buttons

Find (leader only)

- Uses the leader's GPS location. Falls back to Chou Hall if location is blocked.
- Locks everyone's pitch. All phones see a "finding..." screen.
- Pulls up to 60 real restaurants, cafés and fast-food places within about 1.2 km
- Claude returns the pick, a headline, a reason per person, 2 backups, and which places lost and why

Results

- Map of **every** nearby place. The pick is **starred and highlighted**, and losers are struck through.
- 👍 **I'm down** button, with an avatar row showing who's in
- When **everyone** has thumbed up: **directions** in Google Maps or Apple Maps (walking)

5. How Claude does the work

When the leader hits Find, the server sends Claude every pitch (with names), the current Berkeley time, and the restaurant list (id, name, type, cuisine, walk time, opening hours, vegetarian/vegan tags). Claude is told to treat hard constraints (diet, "no X", walking limit, budget, open now) as more important than preferences. It may use general knowledge about well-known places, but has to say "likely" when unsure. It replies with strict JSON:

```
{ "winnerId": "n123", "headline": "Veggie-friendly, cheap, and a 6-minute walk",
  "perPerson": [{ "memberId": "...", "name": "Aisha", "reason": "Tofu pad see ew is cheap and
vegetarian" }],
  "backups":   [{ "id": "n456", "note": "louder, but veggie burritos" }],
  "ruledOut":  [{ "id": "n789", "reason": "Pizza, and Ben vetoed pizza" }] }
```

The server checks that every id actually exists in the list, so Claude can't invent a restaurant. If the reply is bad or there's no API key, it falls back to the closest place.

6. Architecture



Route	What it does
`POST /api/groups`	Create a group with a fun code. The creator becomes the leader.
`GET /api/groups/[code]`	Group status, result and members, plus server time (phones poll this every second)
`POST /api/groups/[code]/member`	Save your name, pitch (≤ 80 chars), typing ping, or thumbs-up. Pitch edits are ignored once Find is hit.
`POST /api/groups/[code]/find`	Leader only, with a lock so it can't run twice: get places, ask Claude, save the result

Data model: `picky:group:CODE` → {leaderId, status: waiting | finding | done, result}. `picky:members:CODE` is a hash with one field per phone → {name, pitch, typingAt, thumb, joinedAt}. Each phone writes only its own field, so friends typing at the same moment never overwrite each other.

7. How it fits the rubric (25 + 2 bonus)

Criterion	How picky answers it
Impact	A real, daily problem for a huge non-technical group (students and friend groups). It saves the 15-minute standoff and ends with everyone agreeing.
Use of Claude	Claude weighs several people's conflicting constraints against real, live place data and explains the trade-off to each person. That's not a chat box: the output drives a map, a vote and directions.
Technical execution	Full end-to-end flow on real phones: live multi-user sync, typing indicators, pitch locking, a leader-only lock on Find, real map data with caching, validated JSON from Claude, and directions.
Demo & storytelling	Every judge has lived "idk, where do you want to eat?" The judges become the friend group and watch it decide for them.
Creativity	A group negotiator rather than a recommendation list, with personality: "dont be picky 😒", live typing, and "why the others lost".
Track fit (+2)	Agents in Action: "AI that finishes the task." It gathers everyone's input, searches real places, decides, gets agreement, and starts directions.

Why the judges should relate

- **Jay Patel** (VoiceBit, AI phone ordering for restaurants): picky helps decide which restaurant gets the order.

- **Ajay Kamat** (Pear VC, consumer and fintech; sold Wedding Party to Instacart): a consumer app about group behavior.
- **Ashtan Jordan** (Precursor; ex-McDonald's product, founded SAMPLEit for discovery): consumer discovery and getting to a choice faster.
- **Ye Wang** (EverCurrent: turning scattered team knowledge into alignment): picky gets a group aligned on one decision.
- **Ayush Malhotra** (Talkhappi, Berkeley Econ & DS, interested in social dynamics): group decisions and social friction.

8. 2-minute demo script

1. **Hook (10 s):** "How long did your group spend deciding where to eat last time? We fixed that."
2. **Hand it over (30 s):** Show the QR code or group code. Each judge joins on their own phone and types one pitch, like "vegetarian", "no pizza" or "under $12". Point out the live "is typing" and try going over the limit to get "dont be picky 😒".
3. **Find (20 s):** Leader taps Find. Every phone switches to the finding screen. The map fills with real nearby restaurants, and one is starred.
4. **Explain (30 s):** Read Claude's one line per judge, then show "why the others lost". The pizza place was crossed out because of the pizza veto.
5. **Close (20 s):** Everyone taps 👍, "Everyone's in 🎉", and the directions button appears. "From 'idk' to walking there in 30 seconds."
6. **Honest limits (10 s):** See the next section.

9. Honest limits & next steps

Limits (say these out loud)

- OpenStreetMap has no prices and sometimes no hours. Claude marks guesses as "likely".
- Phones sync every second (polling), not over websockets.
- No accounts. Groups expire after 6 hours.
- No way to break a tie if someone vetoes the pick yet. You'd re-run Find.

Next

- Google Places or Yelp for prices, ratings and live busyness
- A "veto one" button that re-picks automatically
- Reserve a table or order ahead from the result
- Learn each group's taste over time ("you always end up at Thai")

10. Deploy & run

Deploy to Vercel (about 5 minutes)

1. Import `github.com/SmaranKudapa/picky` into Vercel (Add New → Project).
2. Add the environment variable `ANTHROPIC_API_KEY`.
3. Go to Project → Storage → Create Database → **Upstash for Redis** → Connect. This adds `KV_REST_API_URL` and `KV_REST_API_TOKEN`.
4. Redeploy. Open the URL on your phone and do one practice Find at the demo location. The restaurant list is cached in case the map data service is slow during judging.

Run locally

```
npm install
cp .env.example .env.local      # add ANTHROPIC_API_KEY
npm run dev                     # http://localhost:3000 (in-memory store locally)
PICKY_FAKE_PLACES=1 npm run dev # optional: made-up restaurants, no internet needed
```

Environment variables

Variable	Needed?	Purpose
`ANTHROPIC_API_KEY`	Yes	Claude's pick. Without it, demo mode picks the closest place.
`KV_REST_API_URL` / `KV_REST_API_TOKEN`	Yes on Vercel	Shared live state. Added automatically by Upstash.
`ANTHROPIC_MODEL`	Optional	Override the model (default `claude-sonnet-5-5`)
`PICKY_FAKE_PLACES`	Local testing only	Use made-up restaurants

Project files

File	What's in it
`app/page.js`	The whole phone UI: landing, pitch, waiting room, finding, results, thumbs, directions
`app/MapView.js`	Leaflet map: every place, starred pick, struck-through losers, your location
`app/api/groups/…`	The four API routes above
`lib/pick.js`	Claude prompt, JSON parsing, id validation, fallback
`lib/places.js`	OpenStreetMap Overpass query (3 mirrors), distance and walk time, caching
`lib/store.js`	Upstash Redis, or in-memory for local dev
`lib/config.js`	Pitch limit (80), name limit, default location

Places data © OpenStreetMap contributors. Map tiles © CARTO.